lstmpredict

June 22, 2025

[1]:
```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import re

# --- 1. Prepare Your Data
data = """
Free Fire OB50 update expected end of July 2025.
OB50 Advance Server registration opens in last week of June.
Redeem codes for June 21 include emotes skins and diamonds.
Free Fire MAX group discount event offers up to 90% off bundles.
Free Fire weekly agenda June 18-24 features anniversary backpack.
OB50 update brings Naruto Chapter 2 to the game.
New Solara map launched in OB44 season update.
OB50 Advance Server ends July 20 for early testers.
Redeem codes vary by region and last 24 hours only.
Free Fire MAX top emotes for June include Juggle and Bhangra.
OB49 brought new maps characters and game tweaks in May.
Naruto Shippuden collaboration confirmed in OB50 update.
Exclusive bundles and skins available in group discount event.
Advance Server slots are limited - register quickly.
Daily redeem codes unlock characters and diamonds.
Booyah Pass July 2025 leaked with TV-inspired theme.
Standard Booyah Pass costs 499 diamonds.
Premium bundle costs 999 diamonds for extra rewards.
Free Fire MAX redeem codes highly beneficial, claim fast.
Weekly token roulette on June 21 and Anniversary backpack on June 22.
"""
#source ff faq
corpus = data.lower().split('\n')
corpus = [line.strip() for line in corpus if line.strip()]
corpus = [re.sub(r'[^\w\s]', '', line) for line in corpus]
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(corpus)
total_words = len(tokenizer.word_index) + 1

input_sequences = []
for line in corpus:
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        n_gram_sequence = token_list[:i+1]
        input_sequences.append(n_gram_sequence)

max_sequence_len = max([len(x) for x in input_sequences])
input_sequences = np.array(pad_sequences(input_sequences,␣
 ↪maxlen=max_sequence_len, padding='pre'))

X = input_sequences[:, :-1]
labels = input_sequences[:, -1]
y_true = tf.keras.utils.to_categorical(labels, num_classes=total_words)
```

[2]:
```
# --- 2. Build Your Model ---
model = Sequential()
model.add(Embedding(total_words, 100, input_length=max_sequence_len-1))
model.add(LSTM(150))
model.add(Dense(total_words, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam',␣
 ↪metrics=['accuracy'])
print("Model summary:")
print(model.summary()) # Keep summary as it's informative
```

```
Model summary:

C:\Users\bhara\anaconda3\Lib\site-
packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument
`input_length` is deprecated. Just remove it.
  warnings.warn(

Model: "sequential"


 Layer (type)                        Output Shape                      ␣
 ↪Param #

 embedding (Embedding)               ?                                 0␣
 ↪(unbuilt)

 lstm (LSTM)                         ?                                 0␣
 ↪(unbuilt)
```

```
 dense (Dense)                                ?                                       0␣
↪(unbuilt)


 Total params: 0 (0.00 B)


 Trainable params: 0 (0.00 B)


 Non-trainable params: 0 (0.00 B)
```

```
None
```

[3]:
```
# --- 3. Train Your Model ---
print("\nTraining the model...")
history = model.fit(X, y_true, epochs=100, verbose=0) # Set verbose to 0 for␣
 ↪less output during training
print("Training complete.")

# --- 4. Evaluate & Refine ---
final_loss = history.history['loss'][-1]
final_accuracy = history.history['accuracy'][-1]
print(f"\nModel Evaluation (from training):")
print(f"Final Training Loss: {final_loss:.4f}")
print(f"Final Training Accuracy: {final_accuracy:.4f}")

y_pred_probs = model.predict(X, verbose=0) # Set verbose to 0 for less output␣
 ↪during prediction
rmse = np.sqrt(mean_squared_error(y_true, y_pred_probs))
rae_numerator = np.sum(np.abs(y_pred_probs - y_true))
rae_denominator = np.sum(np.abs(np.mean(y_true) - y_true))
rae = rae_numerator / rae_denominator
r2 = r2_score(y_true, y_pred_probs)

print(f"Root Mean Squared Error (RMSE) on training data: {rmse:.4f}")
print(f"Relative Absolute Error (RAE) on training data: {rae:.4f}")
print(f"R2 Score on training data: {r2:.4f}")
```

```
Training the model…
Training complete.

Model Evaluation (from training):
Final Training Loss: 0.2440
Final Training Accuracy: 0.9438
Root Mean Squared Error (RMSE) on training data: 0.0275
```

```
Relative Absolute Error (RAE) on training data: 0.1850
R2 Score on training data: 0.8540
```

[5]:
```python
# --- 5. Make Predictions ---
def predict_top_n_next_words(seed_text, top_n=3):
    token_list = tokenizer.texts_to_sequences([seed_text])[0]
    token_list = pad_sequences([token_list], maxlen=max_sequence_len-1,␣
 ↪padding='pre')

    predicted_probs = model.predict(token_list, verbose=0)[0]

    top_n_indices = predicted_probs.argsort()[-top_n:][::-1]

    predicted_words_with_probs = []
    for index in top_n_indices:
        output_word = ""
        for word, idx in tokenizer.word_index.items():
            if idx == index:
                output_word = word
                break
        predicted_words_with_probs.append((output_word, predicted_probs[index]))

    return predicted_words_with_probs

print("\n--- Next Word Prediction (Type 'q' to quit) ---")
while True:
    user_input = input("Your text: ").lower()
    if user_input == 'q':
        break
    if not user_input:
        print("Please enter some text.")
        continue

    try:
        top_predictions = predict_top_n_next_words(user_input, top_n=3)
        print(f"Top 3 predicted next words for '{user_input}':")
        for word, prob in top_predictions:
            print(f"- {word} (Probability: {prob:.4f})")
    except Exception as e:
        print("Could not predict words. Your input might contain words not in␣
 ↪the vocabulary.")
        print(f"Error: {e}")

print("\nProgram session ended.")
```

```
--- Next Word Prediction (Type 'q' to quit) ---
```

```
Your text:  q

Program session ended.
```

[ ]: